

Django-2

Django Templates and Dynamic Web Pages

Main goal

By the end of Day 2,

- Why writing HTML directly inside Python is a bad approach.
- What a Django template is.
- How a view renders a template.
- What `render()` does.
- How data moves from Python into HTML.
- What template context is.
- How `{{ variable }}` works.
- How `{% if %}` and `{% for %}` work.
- Why template inheritance exists.
- How `base.html` works.
- How navigation between Django pages works.
- What static files are.
- How Django serves CSS during development.

By the end of the class, the blog should have a proper structure:

```
Home
About
Contact
```

with a shared:

```
Header
Navigation
Page layout
CSS
```

instead of returning raw HTML strings from views.

1. Review Previous Class

What happens when I visit `/about/`?

Browser
↓
GET /about/
↓
Django
↓
URL configuration
↓
about() view
↓
HttpResponse
↓
Browser

Use the existing Day 1 code:

```
def about(request):  
    return HttpResponse(  
        "<h1>About BeginnerBlog</h1>"  
        "<p>We are learning Django.</p>"  
    )
```

Where is our HTML currently written?

Answer:

Inside Python.

What would happen if this page had 300 lines of HTML?

For example:

```
def about(request):  
    return HttpResponse("""  
        <html>  
            <head>  
                <title>About</title>  
            </head>  
            <body>  
                <nav>  
                    ...  
                </nav>  
  
                <main>  
                    ...  
                </main>  
  
                <footer>  
                    ...  
                </footer>  
            </body>  
        </html>
```

```
""")
```

Technically, this could work.

But it creates an important problem.

We are mixing:

```
Python logic
+
HTML presentation
```

inside the same file.

We want different parts of the application to have different responsibilities.

Our view should primarily decide:

```
What needs to happen?
What data is required?
What response should be produced?
```

HTML should primarily describe:

```
What should the page look like?
```

2. Separation of concerns

Write:

```
views.py
Python/backend logic
```

and:

```
HTML template
Page presentation
```

Instead of:

```
View
↓
huge HTML string
```

we want:

```
View
↓
Template
↓
HTML
↓
Browser
```

Django templates allow us to keep HTML in HTML files instead of constructing the page inside Python strings.

This is called **separation of concerns**.

You don't need a formal software-engineering definition yet.

A beginner-friendly definition is:

Different parts of the program should have clearly separated responsibilities.

3. What is a template?

```
<h1>About BeginnerBlog</h1>
<p>We are learning Django.</p>
```

A Django template is essentially:

An HTML file that Django can process before sending it to the browser.

It can contain normal HTML:

```
<h1>Welcome</h1>
```

but also special Django template instructions such as:

```
<h1>Hello {{ name }}</h1>
```

Django can replace:

```
{{ name }}
```

with data supplied by Python.

For example:

Python:

```
name = "Alice"
```

Template:

```
<h1>Hello {{ name }}</h1>
```

Result sent to browser:

```
<h1>Hello Alice</h1>
```

Important distinction:

The browser does not receive `{{ name }}` and figure it out.

Django processes the template **on the server**.

The browser receives normal HTML.

4. Create the templates directory

Inside the `blog` app create:

```
blog/  
└─ templates/  
    └─ blog/
```

Then create:

```
blog/  
└─ templates/  
    └─ blog/  
        ├── home.html  
        ├── about.html  
        └── contact.html
```

Explain why there is another `blog` folder inside `templates`.

It may look strange:

```
blog/templates/blog/
```

A Django project may contain many apps, and different apps might both have a file called `home.html`.

For example:

```
blog/templates/blog/home.html
shop/templates/shop/home.html
```

The inner folder acts as a namespace.

We can therefore refer to our template clearly as:

```
blog/home.html
```

rather than simply:

```
home.html
```

For this course, always use `app_name/templates/app_name/`.

That gives them one consistent pattern.

5. Create the first real template

Create:

```
blog/templates/blog/home.html
```

Add:

```
<!DOCTYPE html>
<html>
<head>
  <title>BeginnerBlog</title>
</head>
<body>

  <h1>Welcome to BeginnerBlog</h1>

  <p>Learning Django one step at a time.</p>

</body>
</html>
```

Nothing Django-specific has happened yet.

This is simply HTML.

Now we need our view to return this file.

6. Introduce `render()`

Day 1 had:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("<h1>Welcome to BeginnerBlog</h1>")
```

Replace it with:

```
from django.shortcuts import render

def home(request):
    return render(request, "blog/home.html")
```

Explain each part.

`render`

Django provides the `render()` shortcut.

Conceptually:

```
render(
    incoming request,
    template to use
)
```

Django:

1. Finds the template.
2. Processes it.
3. Produces HTML.
4. Creates an HTTP response.
5. Returns that response.

So:

```
return render(request, "blog/home.html")
```

can mentally be read as:

Use `blog/home.html` to create the response for this request.

7. Compare `HttpResponse` and `render`

```
return HttpResponse("<h1>Hello</h1>")
```

versus:

```
return render(request, "blog/home.html")
```

`HttpResponse`:

We provide the response content ourselves.

`render`:

Django generates the response content using a template.

Both still eventually result in an HTTP response.

So our Day 1 mental model has not changed.

Previously:

```
Request
↓
View
↓
HttpResponse
↓
Browser
```

Now:

```
Request
↓
View
↓
Template
↓
HttpResponse
↓
Browser
```

The template is not replacing the view.

It is helping the view generate the response.

8. Convert the remaining pages

Create:

about.html

```
<!DOCTYPE html>
<html>
<head>
    <title>About</title>
</head>
<body>

    <h1>About BeginnerBlog</h1>

    <p>
        BeginnerBlog is a project we are building while learning Django.
    </p>

</body>
</html>
```

contact.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Contact</title>
</head>
<body>

    <h1>Contact Us</h1>

    <p>Email: hello@example.com</p>

</body>
</html>
```

Then update:

```
from django.shortcuts import render

def home(request):
    return render(request, "blog/home.html")

def about(request):
    return render(request, "blog/about.html")

def contact(request):
    return render(request, "blog/contact.html")
```

Notice:

`blog/urls.py` does not need to change.

```
urlpatterns = [
    path("", views.home, name="home"),
    path("about/", views.about, name="about"),
    path("contact/", views.contact, name="contact"),
]
```

Why didn't URLs change?

Because the relationship:

URL → view

hasn't changed.

Only how the view creates its response changed.

9. First debugging opportunity: TemplateDoesNotExist

Deliberately type:

```
return render(request, "blog/homee.html")
```

Visit `/`.

Django should produce an error.

Discuss the name:

```
TemplateDoesNotExist
```

What is Django telling us?

Not:

Django is broken.

But:

The view asked Django to load a template it could not find.

Teach beginners to read error messages from the bottom upward and look for:

```
Exception type
File
Line
Message
```

Correct it back to:

```
"blog/home.html"
```

This helps students start treating errors as information.

10. Static HTML is still not enough

Suppose we want the homepage to greet a user by name.

Bad approach:

```
<h1>Hello Alice</h1>
```

Because it is permanently hard-coded.

Another bad approach:

```
return HttpResponse("<h1>Hello " + name + "</h1>")
```

because we are again generating HTML inside Python.

Instead we want:

```
Python provides data
      ↓
Template displays data
```

This introduces **context**.

11. What is template context?

Use a plain Python dictionary first.

Students already know this:

```
student = {
    "name": "Alice",
    "age": 20,
}
```

Django views can pass a dictionary of values to the template.

Example:

```
def home(request):
    context = {
        "name": "Alice",
    }
    return render(request, "blog/home.html", context)
```

Explain the three arguments:

```
render(
    request,
    "blog/home.html",
    context
)
```

Then modify the template:

```
<h1>Welcome, {{ name }}</h1>
```

Browser receives:

```
<h1>Welcome, Alice</h1>
```

Define **context** simply:

Context is the data a view makes available to a template.

12. Trace data through the system

Draw:

```
Browser
  ↓
GET /
  ↓
URL
  ↓
home()
```

↓
context dictionary
{
 "name": "Alice"
}
↓
home.html
↓
{{ name }}
↓
HTML
↓
Browser

Where did Alice originate?

Python.

Where was it displayed?

Template.

Did JavaScript retrieve it?

No.

Did the browser execute Python?

No.

Django generated the HTML before sending it.

13. Django template variables

Use:

```
context = {  
    "name": "Alice",  
    "course": "Django",  
    "lesson_number": 2,  
}
```

Template:

```
<h1>Hello {{ name }}</h1>
```

```
<p>You are learning {{ course }}.</p>
```

```
<p>This is lesson {{ lesson_number }}.</p>
```

Explain:

```
{{ ... }}
```

means:

Output some value here.

Call these **template variables**.

Do not call everything a "template tag."

There are two different syntaxes students need to distinguish.

Variables:

```
{{ name }}
```

Template tags:

```
{% if ... %}  
{% for ... %}
```

14. Passing lists

Students already know Python lists.

Use:

```
def home(request):  
    context = {  
        "topics": [  
            "Django Views",  
            "Templates",  
            "URLs",  
        ],  
    }  
  
    return render(request, "blog/home.html", context)
```

You could technically access:

```
{{ topics.0 }}
```

but don't build the page that way.

This gives us a reason to introduce loops.

15. Template for loops

You know:

```
topics = ["Views", "Templates", "URLs"]

for topic in topics:
    print(topic)
```

Django templates provide their own loop syntax:

```
<ul>

    {% for topic in topics %}

        <li>{{ topic }}</li>

    {% endfor %}

</ul>
```

Explain carefully:

```
{% for topic in topics %}
```

starts the loop.

```
{{ topic }}
```

displays the current value.

```
{% endfor %}
```

ends the loop.

Unlike Python:

```
for topic in topics:
```

Django templates don't use indentation to determine where the loop ends.

They explicitly use:

```
{% endfor %}
```

16. Why doesn't Django let us just write Python in HTML?

Templates are intended mainly for presentation, not general application logic.

We want complicated logic in Python.

For example:

```
views.py
```

should decide which data the page needs.

The template should mostly decide:

How should that data be displayed?

This keeps responsibilities clearer.

17. Template `if`

Pass:

```
context = {
    "name": "Alice",
    "is_student": True,
}
```

Template:

```
{% if is_student %}

    <p>Welcome, student!</p>

{% else %}

    <p>Welcome, visitor!</p>

{% endif %}
```

Relate it to Python:

```
if is_student:
    ...
else:
    ...
```

But point out the template syntax:

```
{% if %}
{% else %}
{% endif %}
```

Again:

```
{{  }}
```

outputs values.

```
{% %}
```

performs template instructions.

That distinction should be drilled repeatedly.

18. A realistic temporary blog example

We do not have a database yet.

That's tomorrow.

But we can simulate blog posts with Python data.

Use:

```
def home(request):
    posts = [
        {
            "title": "My First Post",
            "author": "Alice",
        },
        {
            "title": "Learning Django",
            "author": "Bob",
        },
        {
            "title": "Why Python Is Fun",
            "author": "Charlie",
        },
    ],
```

```
]

context = {
    "posts": posts,
}

return render(request, "blog/home.html", context)
```

Then:

```
<h1>Latest Posts</h1>

{% for post in posts %}

    <article>

        <h2>{{ post.title }}</h2>

        <p>Written by {{ post.author }}</p>

    </article>

{% endfor %}
```

Students may notice:

```
post["title"]
```

in Python has become:

```
{{ post.title }}
```

Explain:

Django template syntax uses dot notation for looking up values in structures like dictionaries and objects.

Don't go deeply into Django's variable lookup algorithm.

For beginners:

```
Python:
post["title"]
```

```
Template:
{{ post.title }}
```

is enough.

19. Empty lists

Change:

```
posts = []
```

What happens?

Nothing appears.

Improve it:

```
{% if posts %}

    {% for post in posts %}

        <article>
            <h2>{{ post.title }}</h2>
            <p>{{ post.author }}</p>
        </article>

    {% endfor %}

{% else %}

    <p>No posts are available yet.</p>

{% endif %}
```

Ask:

Which logic belongs here?

This is simple presentation logic:

If posts exist, display them. Otherwise display a message.

That's reasonable template logic.

20. Identify a new problem: duplicated HTML

At this point we have:

```
home.html
about.html
contact.html
```

And every file may contain:

```
<!DOCTYPE html>
<html>
<head>
...
</head>
<body>
...
</body>
</html>
```

Now introduce a navbar:

```
<nav>
  <a href="/">Home</a>
  <a href="/about/">About</a>
  <a href="/contact/">Contact</a>
</nav>
```

Ask:

Where do we put this?

If we put it into all three files, it works.

But now imagine changing:

Contact

to:

Contact Us

You would need to change every HTML file.

With:

100 pages

you might have to update 100 files.

Introduce:

Template inheritance solves this duplication problem.

21. Template inheritance

Create:

blog/templates/blog/base.html

Use:

```
<!DOCTYPE html>
<html>

<head>

    <title>BeginnerBlog</title>

</head>

<body>

    <nav>

        <a href="/">Home</a>

        <a href="/about/">About</a>

        <a href="/contact/">Contact</a>

    </nav>

    <main>

        {% block content %}
        {% endblock %}

    </main>

</body>

</html>
```

Explain:

base.html contains things shared across pages.

For example:

```
HTML structure
navigation
header
footer
```

CSS references

The special part is:

```
{% block content %}  
{% endblock %}
```

Think of this as:

Child pages may place their page-specific content here.

22. Extend the base template

Change:

home.html

to:

```
{% extends "blog/base.html" %}  
  
{% block content %}  
    <h1>Welcome to BeginnerBlog</h1>  
    <p>Learning Django one step at a time.</p>  
{% endblock %}
```

Now explain:

```
{% extends "blog/base.html" %}
```

means:

Use `base.html` as the overall structure for this page.

Then:

```
{% block content %}  
...  
{% endblock %}
```

means:

Put this content into the `content` block defined by the parent.

Conceptually:

```
base.html

Navigation

[ CONTENT GOES HERE ]

Footer
```

Home provides:

```
Welcome to BeginnerBlog
```

About provides:

```
About BeginnerBlog
```

Contact provides:

```
Contact Us
```

23. Update all templates

about.html

```
{% extends "blog/base.html" %}

{% block content %}

    <h1>About BeginnerBlog</h1>

    <p>
        BeginnerBlog is a project we are building while learning Django.
    </p>

{% endblock %}
```

contact.html

```
{% extends "blog/base.html" %}

{% block content %}

    <h1>Contact Us</h1>
```

```
<p>Email: hello@example.com</p>

{% endblock %}
```

Ask:

Where is the `<html>` element now?

Only in:

```
base.html
```

Where is navigation?

Only in:

```
base.html
```

Where is page-specific content?

Inside each page's block.

24. Why inheritance is useful

Before:

```
home.html
├── HTML structure
├── nav
└── content
```

```
about.html
├── HTML structure
├── nav
└── content
```

```
contact.html
├── HTML structure
├── nav
└── content
```

After:

```
base.html
├── HTML structure
└── nav
```

```
home.html
```

```
└─ home content

about.html
└─ about content

contact.html
└─ contact content
```

This demonstrates the general programming idea:

Avoid unnecessary duplication.

25. Improve the page title using blocks

Currently:

```
<title>BeginnerBlog</title>
```

is identical everywhere.

We can create another block.

In `base.html`:

```
<title>
    {% block title %}
        BeginnerBlog
    {% endblock %}
</title>
```

Then `about.html`:

```
{% extends "blog/base.html" %}

{% block title %}
    About | BeginnerBlog
{% endblock %}

{% block content %}

    <h1>About BeginnerBlog</h1>

{% endblock %}
```

A base template can expose several areas that child templates may replace.

For now:

```
title  
content
```

is enough.

Don't create unnecessarily complicated inheritance hierarchies.

26. Stop hard-coding URLs

Currently:

```
<a href="/about/">About</a>
```

works.

But ask:

What happens if later we change `/about/` to `/about-us/`?

Every manually written link might need changing.

Remember Day 1:

```
path("about/", views.about, name="about")
```

We deliberately gave it a name:

```
name="about"
```

Now students discover why.

Django templates can refer to that name.

Replace:

```
<a href="/about/">About</a>
```

with:

```
<a href="{% url 'about' %}">About</a>
```

Use:

```
<nav>

    <a href="{% url 'home' %}">Home</a>

    <a href="{% url 'about' %}">About</a>

    <a href="{% url 'contact' %}">Contact</a>

</nav>
```

Explain:

We are telling Django, "give me the URL whose name is `about`."

This separates:

what the URL is called

from:

what its path currently looks like

27. Demonstrate why named URLs matter

Change:

```
path("about/", views.about, name="about")
```

to:

```
path("about-us/", views.about, name="about")
```

Without changing:

```
{% url 'about' %}
```

refresh the page.

Navigation now points to:

```
/about-us/
```

This is much more convincing than simply telling students named URLs are good.

28. Introduce static files

Now ask:

Where should CSS live?

Not inside:

```
views.py
```

and ideally not repeated inside every template.

CSS doesn't usually change for every request.

Files such as:

```
CSS
JavaScript
images
fonts
```

are called **static files**.

Explain:

A template is processed by Django to generate a response. Static files are files that can be served largely as they are.

For today's lesson:

```
HTML → templates
CSS → static files
```

29. Create the static directory

Create:

```
blog/
├── static/
│   └── blog/
│       └── styles.css
```

Again explain the namespace:

```
static/blog/styles.css
```

rather than simply:

```
static/styles.css
```

because multiple Django apps may contain their own static files.

30. Add simple CSS

Use deliberately simple CSS:

```
body {
    font-family: Arial, sans-serif;
    max-width: 800px;
    margin: 40px auto;
    padding: 0 20px;
}

nav {
    margin-bottom: 30px;
}

nav a {
    margin-right: 15px;
}

article {
    border-bottom: 1px solid #ddd;
    padding: 15px 0;
}
```

The goal isn't teaching CSS.

The goal is understanding how Django includes static assets.

31. Load static files

At the top of:

```
base.html
```

write:

```
{% load static %}
```

Then:

```
<link rel="stylesheet" href="{% static 'blog/styles.css' %}">
```

Full beginning:

```
{% load static %}

<!DOCTYPE html>

<html>

<head>

    <title>
        {% block title %}
            BeginnerBlog
        {% endblock %}
    </title>

    <link
        rel="stylesheet"
        href="{% static 'blog/styles.css' %}"
    >

</head>
```

Explain:

```
{% load static %}
```

makes Django's static template functionality available.

Then:

```
{% static 'blog/styles.css' %}
```

asks Django for the appropriate URL for that static file.

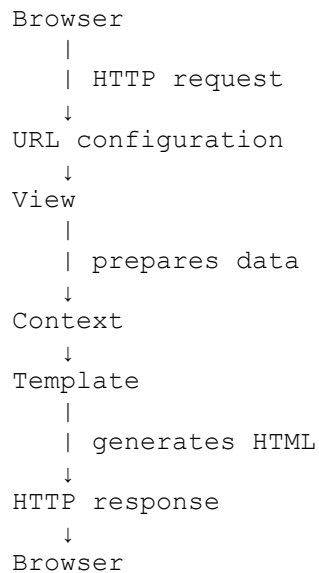
Students don't need to understand production static-file hosting yet.

Simply explain:

During development, Django helps us locate and serve these static files. Production deployment handles static files differently, which we'll discuss later.

32. Updated architecture after Day 2

Now expand Day 1's diagram.



And separately:



This should become today's central mental model.

33. What we still don't have

Ask:

Where are our blog posts currently coming from?

Answer:

A Python list:

```
posts = [
    ...
]
```

What's wrong with that?

If we stop the server or edit the code:

the "posts" aren't real saved application data

We need persistent storage.

Lead into Day 3:

Today:

Python list

↓

Template

Tomorrow:

Database

↓

Django Model

↓

View

↓

Template

This gives a natural reason for learning models rather than introducing databases randomly.

Day 2 completed code

blog/views.py

```
from django.shortcuts import render
```

```
def home(request):
```

```
    posts = [
        {
            "title": "My First Post",
            "author": "Alice",
        },
        {
            "title": "Learning Django",
            "author": "Bob",
        },
        {
            "title": "Why Python Is Fun",
            "author": "Charlie",
        },
    ]
```

```
    context = {
        "posts": posts,
    }
```

```
        return render(request, "blog/home.html", context)

def about(request):
    return render(request, "blog/about.html")

def contact(request):
    return render(request, "blog/contact.html")
```

blog/urls.py

```
from django.urls import path

from . import views

urlpatterns = [
    path("", views.home, name="home"),
    path("about/", views.about, name="about"),
    path("contact/", views.contact, name="contact"),
]
```

base.html

```
{% load static %}

<!DOCTYPE html>

<html>

<head>

    <title>
        {% block title %}
            BeginnerBlog
        {% endblock %}
    </title>

    <link
        rel="stylesheet"
        href="{% static 'blog/styles.css' %}"
    >

</head>

<body>

    <nav>

        <a href="{% url 'home' %}">
```

```
        Home
    </a>

    <a href="{% url 'about' %}">
        About
    </a>

    <a href="{% url 'contact' %}">
        Contact
    </a>

</nav>

<main>

    {% block content %}
    {% endblock %}

</main>

</body>

</html>
```

home.html

```
{% extends "blog/base.html" %}

{% block title %}
    Home | BeginnerBlog
{% endblock %}

{% block content %}

    <h1>BeginnerBlog</h1>

    <p>Learning Django one step at a time.</p>

    <h2>Latest Posts</h2>

    {% if posts %}

        {% for post in posts %}

            <article>

                <h3>{{ post.title }}</h3>

                <p>
                    Written by {{ post.author }}
                </p>
            </article>
        {% endfor %}
    {% endif %}

{% endblock %}
```

```
        </p>

    </article>

    {% endfor %}

    {% else %}

        <p>No posts are available yet.</p>

    {% endif %}

{% endblock %}
```

about.html

```
{% extends "blog/base.html" %}

{% block title %}
    About | BeginnerBlog
{% endblock %}

{% block content %}

    <h1>About BeginnerBlog</h1>

    <p>
        BeginnerBlog is a project we are building while learning Django.
    </p>

{% endblock %}
```

contact.html

```
{% extends "blog/base.html" %}

{% block title %}
    Contact | BeginnerBlog
{% endblock %}

{% block content %}

    <h1>Contact Us</h1>

    <p>Email: hello@example.com</p>

{% endblock %}
```

styles.css

```
body {
    font-family: Arial, sans-serif;
    max-width: 800px;
    margin: 40px auto;
    padding: 0 20px;
}

nav {
    margin-bottom: 30px;
}

nav a {
    margin-right: 15px;
}

article {
    border-bottom: 1px solid #ddd;
    padding: 15px 0;
}
```

Day 2 exercise 1 — Add a new page

Students should add:

```
/resources/
```

The page should have:

Resources

Useful resources for learning Django will appear here.

Requirements:

- Create a view.
- Create a URL.
- Give the URL a name.
- Create a template.
- Extend `base.html`.
- Add Resources to the navigation.

They should not duplicate the full HTML document.

Day 2 exercise 2 — Pass data from the view

Modify the resources view.

Python:

```
resources = [  
    "Django Documentation",  
    "Python Documentation",  
    "MDN Web Docs",  
]
```

Pass it to the template.

Students must display:

Resources

- Django Documentation
- Python Documentation
- MDN Web Docs

using:

```
{% for %}
```

Day 2 exercise 3 — Empty-state handling

Change:

```
resources = []
```

The page should show:

No resources are available yet.

Students must use:

```
{% if %}
```

rather than solving it entirely in Python.

Day 2 exercise 4 — Fix the duplication

Give students these templates:

```
page1.html  
page2.html  
page3.html
```

where each contains identical navigation.

Ask:

What design problem do you see?

Expected:

Duplicated structure.

Then:

What Django feature should solve it?

Expected:

Template inheritance.

Day 2 debugging exercise

Give:

```
{% extend "blog/base.html" %}
```

Ask what is wrong.

Expected:

```
{% extends "blog/base.html" %}
```

Then give:

```
{{ for post in posts }}
```

Expected:

```
{% for post in posts %}
```

Then:

```
{% for post in posts %}
```

```
    {{ post.title }}  
{% endif %}
```

Expected:

```
{% endfor %}
```

This is useful because beginners frequently confuse:

```
{{  }}
```

and:

```
{% %}
```

Day 2 vocabulary

Students should be able to explain these terms.

Template

An HTML file that Django can process to produce part or all of a response.

render ()

A Django shortcut that processes a template and returns an HTTP response.

Context

Data passed from a view to a template.

Template variable

A value displayed with syntax such as:

```
{{ name }}
```

Template tag

An instruction such as:

```
{% if %}
```

or:

```
{% for %}
```

Template inheritance

A way for templates to reuse common layout from another template.

Base template

A template containing common page structure shared by other templates.

Block

A replaceable section of a template.

Example:

```
{% block content %}  
{% endblock %}
```

Static file

A file such as CSS, JavaScript, or an image that isn't normally generated dynamically for every request.

Named URL

A URL pattern given a name such as:

```
name="about"
```

so other parts of the application can refer to it without hard-coding its path.

Day 2 concept questions

1. Does the browser execute the Django template?

No.

Django processes the template on the server and sends resulting HTML to the browser.

2. What is the responsibility of a view?

Handle a request, perform or coordinate necessary application logic, prepare data, and produce a response.

3. What is the responsibility of a template?

Describe how data should be presented as HTML.

4. What is context?

Data passed by a view to a template.

5. What is the difference between:

```
{{ name }}
```

and:

```
{% if name %}
```

```
{{ }}:
```

Output a value.

```
{% %}:
```

Execute a template instruction.

6. Why use `base.html`?

To place common page structure in one location and avoid repeating it across many templates.

7. Why use:

```
{% url 'about' %}
```

instead of:

```
href="/about/"
```

Because the template references the URL by its Django name instead of depending directly on the current path.

8. Why does CSS go in static files?

Because CSS is an asset rather than server-side application logic or dynamically generated template content.

9. Are our blog posts currently stored permanently?

No.

They currently exist only as data created inside our Python code.

10. What will solve this next?

A database.

That becomes Day 3.

Day 2 homework

Task 1

Create:

```
/categories/
```

Pass this Python list from the view:

```
categories = [  
    "Python",  
    "Django",  
    "Web Development",  
]
```

Render it as an HTML list.

Task 2

If:

```
categories = []
```

display:

```
No categories have been added.
```

Task 3

Make every page use `base.html`.

No page other than `base.html` should contain:

```
<html>  
<head>  
<body>
```

Task 4

Add a footer to `base.html`:

```
BeginnerBlog — Built while learning Django
```

The footer must automatically appear on every page.

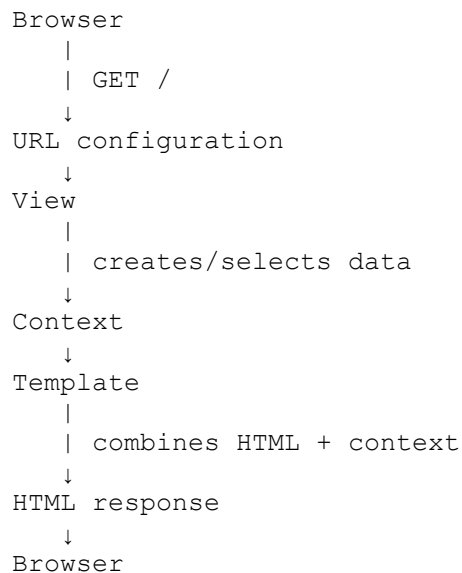
Task 5

Explain in their own words:

1. Why are templates preferable to writing HTML in `HttpResponse()`?
 2. What does `render()` do?
 3. What is context?
 4. What's the difference between `{{ }}` and `{% %}`?
 5. What problem does template inheritance solve?
 6. What is `base.html`?
 7. Why should URLs be referred to by name?
 8. What is a static file?
-

What students should be able to draw from memory

At the end of Day 2, they should be able to draw:



This is the conceptual foundation needed for Day 3.

On Day 3 we will replace this:

```
posts = [
    {
        "title": "My First Post",
        "author": "Alice",
    }
]
```

with actual persistent data:

Database
↓
Post model
↓
View
↓
Template

That is where students will learn:

- What a database actually is.
- Tables, rows and columns.
- Primary keys.
- Django models.
- Model fields.
- ORM.
- Migrations.
- `makemigrations`.
- `migrate`.
- Django admin.
- Creating actual `Post` records.